The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Appendix

A

The Stack: Setting It Up and Keeping It

Taught in —

Laboratory —

Assessment —

This is a class extract. It is *Appendix A* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Chapter A

The Stack: Setting It Up and Keeping It

Everything in this book from Chapter 5 onward assumes a shop you can reach in a browser. This appendix gets you one, in a single sitting, and lists what to do when it refuses — which on a university laptop it sometimes will.

A.1 What You Install

One thing: a container runtime. Not a web server, not PHP, not a database. Those three run inside containers and never touch your own machine, which is why the shop is identical on every laptop in the class and why uninstalling it is one command.

- **Windows or macOS:** Docker Desktop, from `docker.com`. On Windows it needs WSL 2, which its installer offers to set up.
- **Linux:** the `docker.io` and `docker-compose-v2` packages from your distribution, then add yourself to the `docker` group and log out and in again.

Check it with:

```
docker --version
docker compose version
docker run --rm hello-world
```

The third is the one that matters. The first two only prove the program is installed; the third proves the *engine* is running, which is a different claim and the one that usually fails.

A.2 Starting the Shop

```
cd code
cp .env.example .env
docker compose up -d --build
python waitfor.py
```

The first run takes a few minutes and downloads about 1.5 GB; later runs take seconds. Chapter 5 measures both and explains why the difference matters.

What	Where	Sign in with
The storefront	`http://localhost:8090/`	nobody; you are a customer
The admin panel	`http://localhost:8090/admin/`	`admin` / `admin123`
The database console	`http://localhost:8091/`	server `db`, user `shop`, password `shoppw`, database `opencart`

Those credentials are deliberately bad

`admin` / `admin123` is exactly the kind of credential Chapter 16 goes looking for and finds. It is the default so that you can measure the shop before and after hardening it; Chapter 16 asks you to change it, and until then every student in the class can sign in to every other student's shop if they can reach it.

Which they cannot, because nothing here is published to the internet. If you ever expose this stack beyond your own machine, change the credentials in `.env` *first*.

A.3 The Four Commands Worth Memorising

Command	What it does
`docker compose up -d`	Start everything. Safe to run when it is already running
`docker compose down`	Stop everything. **Your shop is kept**
`docker compose logs -f shop`	Watch the shop's log. This is where PHP errors go, and they go nowhere else
`docker compose down -v`	Stop everything **and delete your shop**. The reset button, and the one mistake with no undo

A.4 When It Will Not Start

Five failures account for nearly all of them, and four are not really about Docker.

1. **"Cannot connect to the Docker daemon"** — the engine is not running. On Windows and macOS, start Docker Desktop and wait for its icon to stop animating; the command line is ready several seconds before the whale stops moving.

2. **"Ports are not available"** or "address already in use" — something else on your machine already holds port 8090 or 8091. This is common; the development machine for this book had both 8080 and 8081 taken, which is why the defaults are 8090 and 8091 in the first place. Edit `.env`, change `SHOP_PORT`, and bring it up again. Change only the port in `.env` — `OC_HTTP_SERVER` reads the same variable, so links keep working.

3. **The shop answers but every page is an error** — the install did not finish. Read `docker compose logs shop` and look for "SUCCESS! OpenCart successfully installed". If it is absent, the database was not ready; `docker compose down -v` and start again.

4. **WSL 2 is not installed, or virtualisation is disabled** — Windows only. Docker Desktop's installer handles the first; the second is a BIOS setting (often called "Intel VT-x", "AMD-V" or "SVM Mode") that some laptops ship disabled.

5. **It worked yesterday and today it does not** — almost always disk space. `docker system df` will tell you; `docker system prune` reclaims what is not in use. It does not touch named volumes, so your shop survives — but read what it proposes before agreeing.

Read the log before changing anything

The single most useful habit in this appendix. A shop that is broken is broken for a reason, and the reason is almost always written down in `docker compose logs shop`.

Restarting, rebuilding and resetting are the three things students try first, and all three destroy the evidence. Look at the log, then act.

A.5 Backing Up, and Why You Will Wish You Had

Your shop lives in two named volumes. `down -v` deletes them, and so does an afternoon of clearing disk space without reading the prompt.

```
# Dump the database into the repository, where git can see it.
docker compose exec -T db mariadb-dump -ushop -pshoppw opencart \
  > backups/shop-$(date +%Y%m%d).sql

# Put it back.
```

```
docker compose exec -T db mariadb -ushop -pshoppw opencart \
  < backups/shop-20261001.sql
```

Take one after every chapter's laboratory. It costs two seconds, and Chapter 20 measures what the alternative costs.

Where to keep your own work

Anything you write that must survive belongs in `code/`, in the repository — not inside a running container, where it is deleted at the next rebuild, and not only in the admin panel, where it is deleted by `down -v`.

This matters most in Chapter 19, where you write a module. Write it in `code/modules/` and mount it; do not edit it in place inside the container. Students who learn this in Week 14 learn it the expensive way.

A.6 The Machine Every Measurement Was Taken On

Every figure in a **Measured** box in this book was produced on this machine. Yours will differ; the shapes and ratios should not.

Item	Value
Host	Windows 11 Pro, 4 CPU cores, 7.7 GiB available to the container runtime
Container runtime	Docker 28.1.1, Linux containers
Web and application	Apache 2.4.68, PHP 8.2.34 with OPcache enabled
Database	MariaDB 11.8.9
Application	OpenCart 4.1.0.4, installed from the pinned release in `code/shop/Dockerfile`
Everything on	one machine, so no figure here includes real network latency

The scope of Chapter 16's security work

Chapter 16 asks you to attack a shop: SQL injection, cross-site scripting, missing security headers, default credentials, an exposed administrative path. Every one of those exercises is scoped, in the chapter and here, to **the container running on your own machine, reached at `localhost`, which you installed and which contains no real person's data**.

That is the whole of it. Running the same checks against any system you do not own and have not been authorised in writing to test is a criminal offence in Pakistan under the Prevention of Electronic Crimes Act 2016, and a sackable one nearly everywhere else. The skill is valuable precisely because it is dangerous, and the boundary is not a formality.

Chapter 22 returns to this with the statute in front of it.